

THE TRANSPORT LAYER

- The Transport Service
- Elements of Transport Protocols
- The Internet Transport Protocols (Internet): UDP

THE TRANSPORT SERVICE

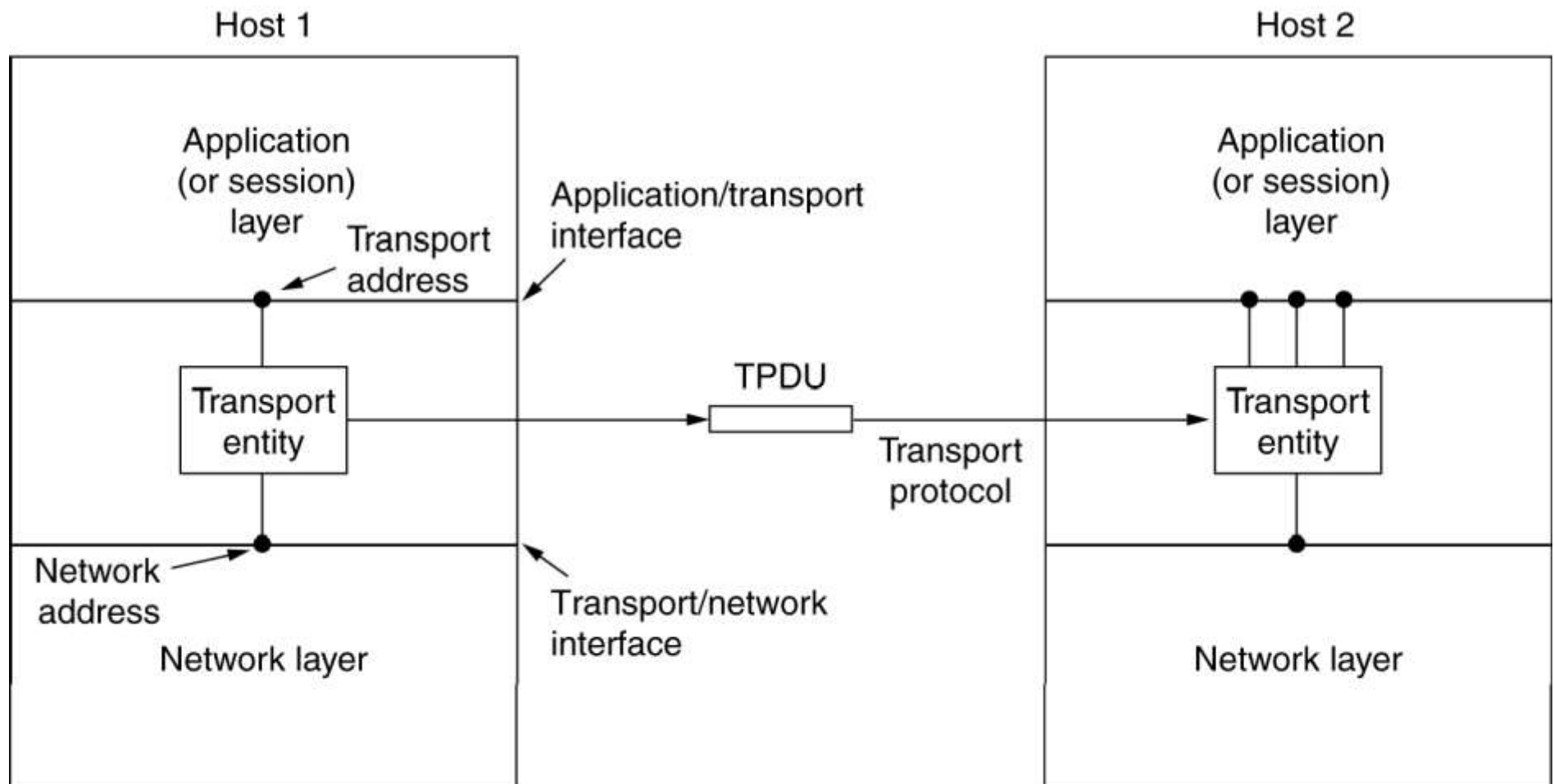
- Services Provided to the Upper Layers
- Transport Service Primitives
- Berkeley Sockets
- An Example of Socket Programming:
 - An Internet File Server

The Transport Service: Services

- **Transport layer services :**
 - To provide efficient, reliable, and cost-effective service to its users, normally processes in the application layer.
 - To make use of the services provided by the network layer.
- **The transport entity:** the hardware and/or software within the transport layer that does the work. Its positions:
 - In the OS kernel, in a separate user process, in a library package bound to network applications, or
 - On the network interface card.

The Transport Service: Services

The network, transport and application layers



The Transport Service: Services

- There are two types transport services
 - Connection-oriented transport service
 - Connectionless transport service
- The similarities between transport services and network services
 - The connection-oriented service is similar to the connection-oriented network service in many ways:
 - Establishment, → data transfer, → release;
 - Addressing;
 - Flow control
 - The connectionless transport service is also similar to the connectionless network service.

The Transport Service: Services

- The differences between transport services and network services. Why are there two distinct layers?
 - The transport code runs entirely on the user's machines, but the network layer mostly runs on the routers which are usually operated by the carrier.
 - Network layer has problems (losing packets, router crashing, ...)
 - The transport layer improves the QOS of the network layer.
 - → The transport service is more reliable than the network service.
 - → Applications programmers can write code according to a standard set of transport service primitives and have these programs work on a wide variety of networks.

The Transport Service: Services

- The transport layer provides some operations to applications programs, i.e., a transport service interface.
- Each transport service has its own interface.
- The transport service is similar to the network service, but there are also some important differences:
 - Network service models real network → unreliable.
Transport services improves real network → reliable.
 - Network services are for network developers.
Transport services are for application developers.

The Transport Service: Hypothetical Primitives

Primitive	Packet sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA packet arrives
DISCONNECT	DISCONNECTION REQ.	This side wants to release the connection

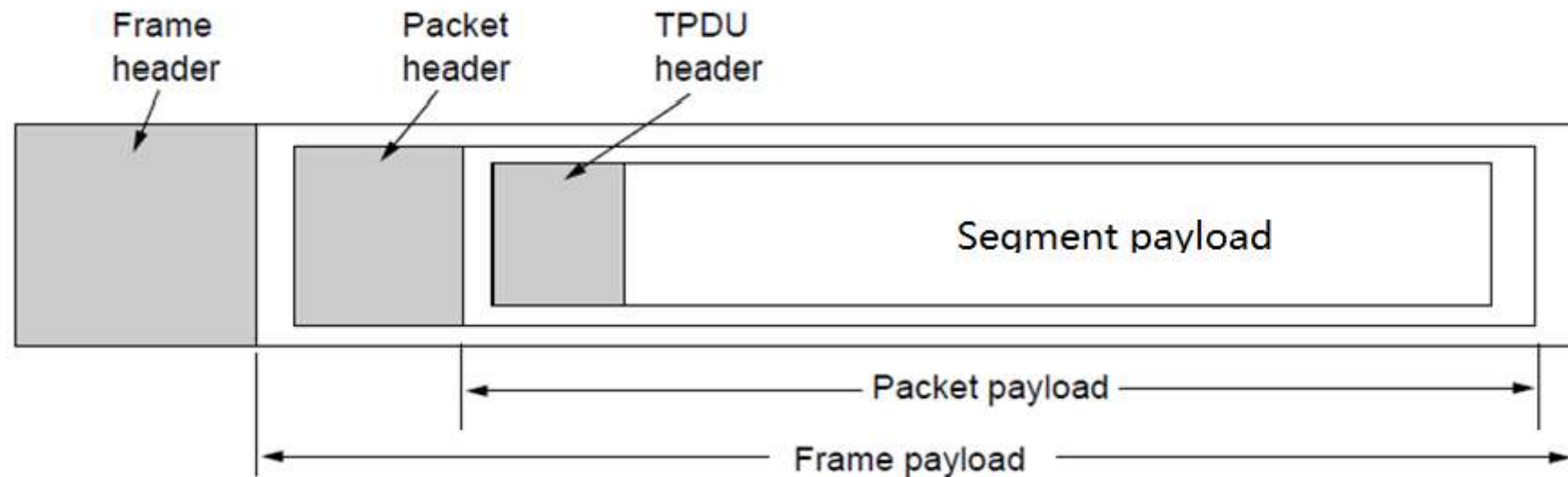
The Transport Service: Hypothetical Primitives

How to use these primitives for an application?

- Server App: LISTEN
- Client App: CONNECT
- Client ←SEND/RECEIVE→ Server
- Client Transport ← TPDU → Server Transport
 - Client network layer ← Packets→ Server network layer
 - Client data link layer ← Frames → Server data link layer
- Client/Server: DISCONNECT

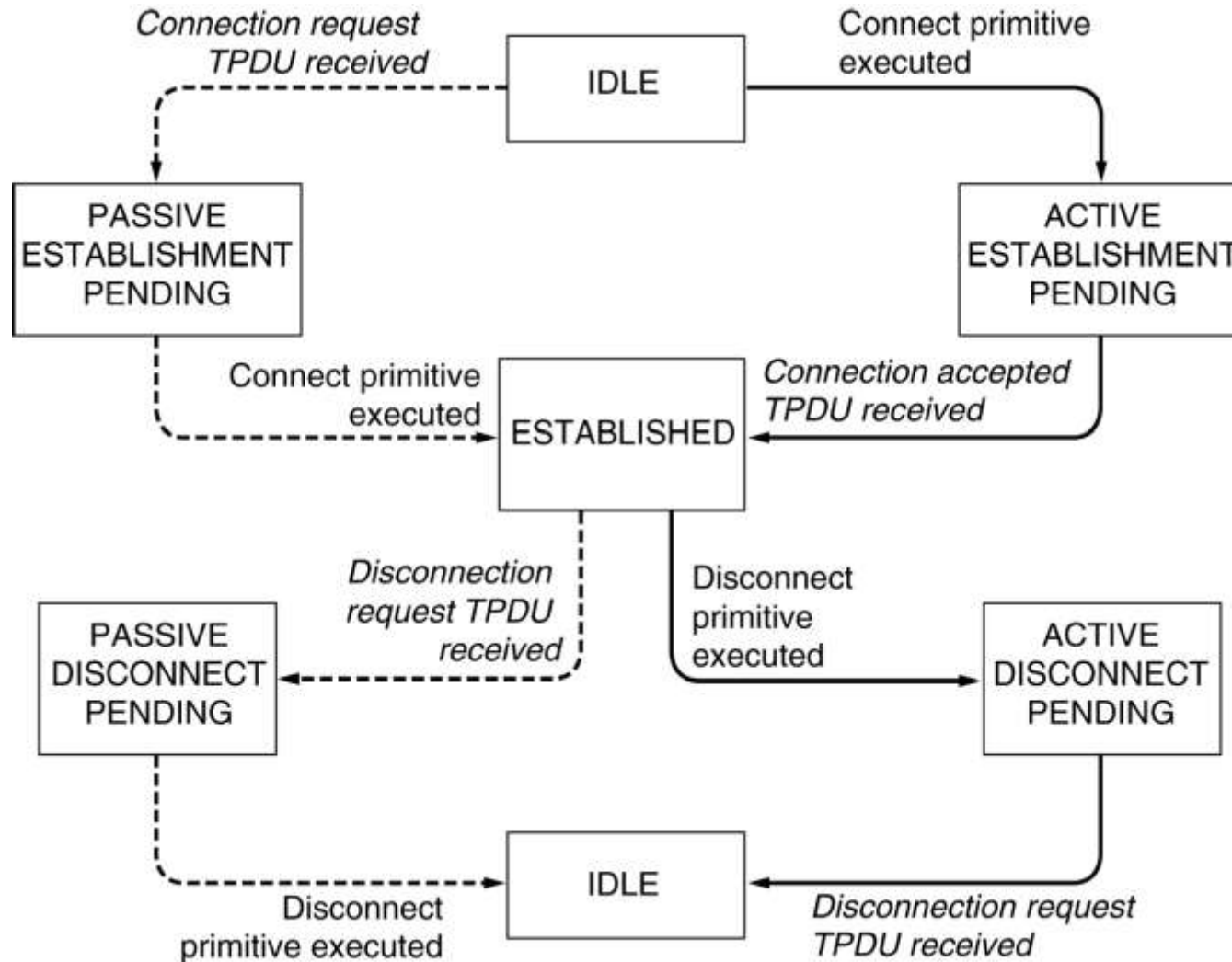
The Transport Service: Hypothetical Primitives

Nesting of Segments, packets, and frames



The Transport Service: Hypothetical Primitives

Connection establishment and connection release



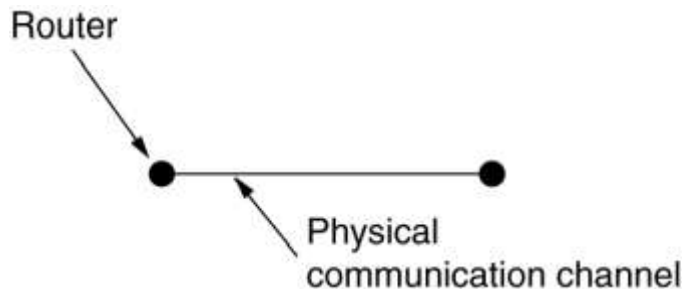
ELEMENTS OF TRANSPORT PROTOCOLS

- Addressing
- Connection Establishment
- Connection Release
- Error Control and Flow Control
- Multiplexing
- Crash Recovery

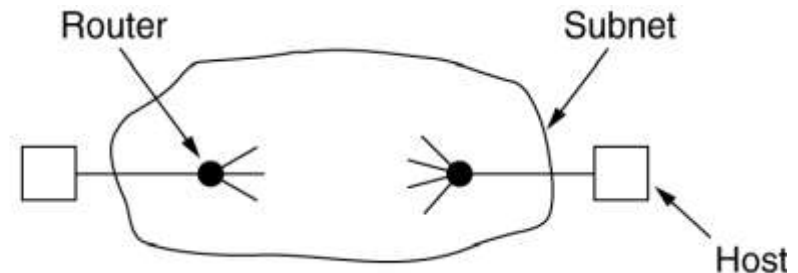
Elements of Transport Protocols

Transport protocol and data link protocol

- Similarities: Error control, sequencing, and flow control
- Differences
 - Environment: physical communication channel/subnet
 - Addressing: implicit/explicit
 - Connection establishment: simple/complicated
 - Storage capacity: no/yes (unpredictable/predictable)
 - Buffering and flow control: amount



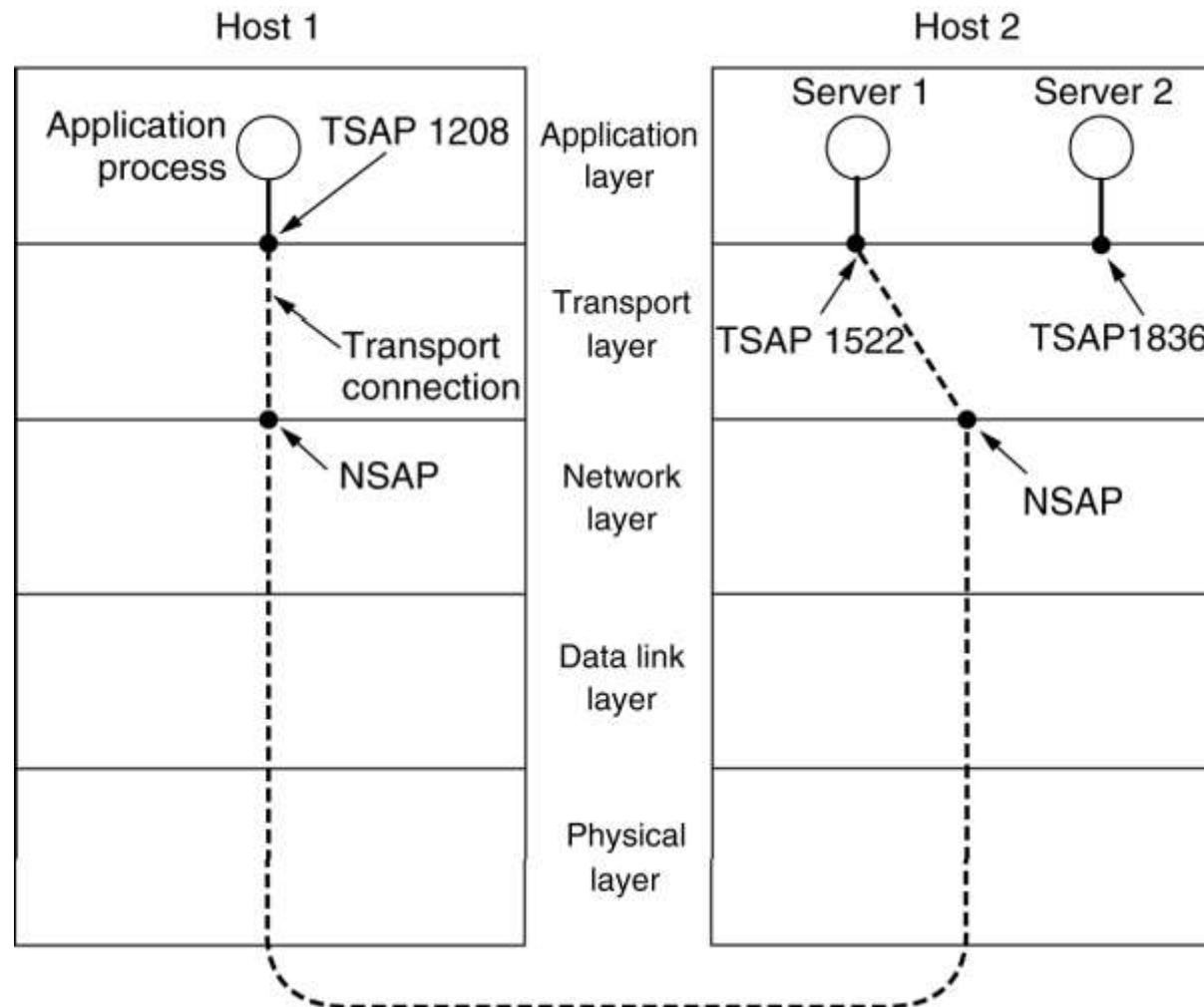
(a)



(b)

Elements of Transport Protocols: Addressing

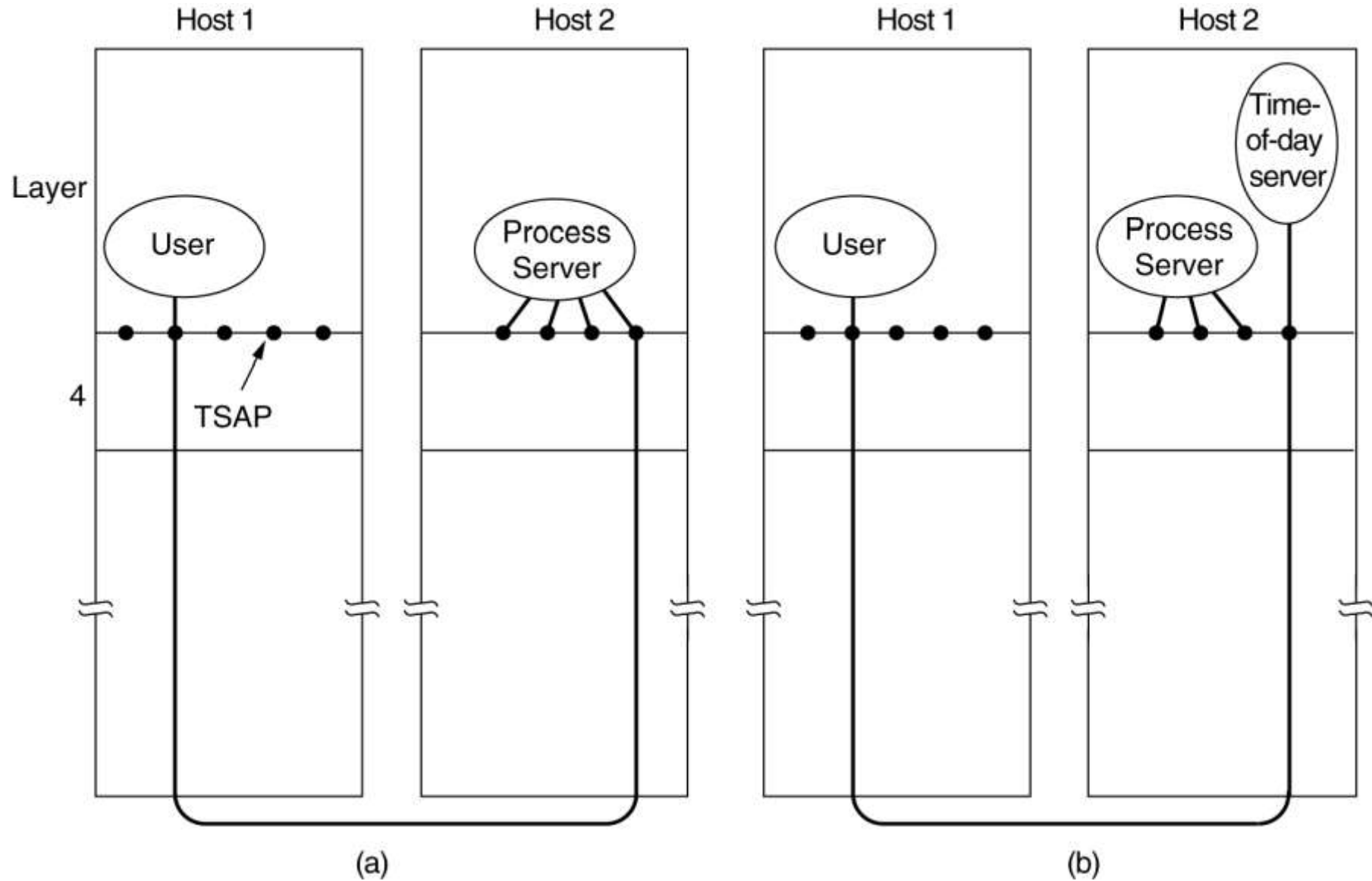
How to address?



Elements of Transport Protocols: Addressing

- NSAP: Network service access points
 - IP (32 bits)
- TSAP: Transport service access point
 - Port (16 bits)
- How to find out the server's TSAP
 - Some TSAP addresses are so famous that they are fixed.
 - Process server
 - To listen to a set of TSAPs at the same time.
 - To dispatch the request to the right server.
 - Process server + name server (directory server, 114)

Elements of Transport Protocols: Addressing



Elements of Transport Protocols:

Connection Establishment

Connection establishment sounds easy, but ...

- Naïve approach:
 - One sends a CONNECTION REQUEST TPDU to the other and wait for a CONNECTION ACCEPTED reply.
 - If ok, done; otherwise, retry.
 - Possible nightmare:
 - A user establishes a connection with a bank, sends messages telling the bank to transfer a large amount of money to the account of a not-entirely-trustworthy person, and then releases the connection.
 - Moreover, assume each packet in scenario is ***duplicated*** and stored in the subnet.
 - → ***delayed duplicates***.

Elements of Transport Protocols:

Connection Establishment

Possible solutions for solving *delayed duplicates*

- To give each connection a connection identifier (i.e., a sequence number incremented for each connection established) chosen by the initiating party and put in each TPDU, including the one requesting the connection.
- To use throw away transport addresses. Each time a transport address is needed, a new one is generated. When a connection is released, the address is discarded and never used again.
- To use sequence number and age
 - To limit packet lifetime.
 - To use the sequence number

Elements of Transport Protocols:

Connection Establishment

- Packet lifetime can be restricted to a known maximum using one of the following techniques
 - Restricted subnet design.
 - Putting a hop counter in each packet.
 - Timestamping each packet. (router synchronization problem)
- In practice, **we will need to guarantee not only that a packet is dead, but also that all acknowledgements to it are also dead**, so we will now introduce T , which is some small multiple of the true maximum packet lifetime.

Elements of Transport Protocols: Connection Establishment

To ensure that two identically numbered TPDUs are never outstanding at the same time:

1. To equip each host with a time-of-day clock
 - Each clock is assumed to take the form of a binary counter that increments itself at uniform intervals.
 - The number of bits in the counter must equal or exceed the number of bits in the sequence numbers.
 - The clock is assumed to continue running even if the host goes down.
 - The clocks at different hosts need not be synchronized.

Elements of Transport Protocols: Connection Establishment

To ensure that two identically numbered TPDUs are never outstanding at the same time:

2. When a connection is setup, the low order k bits of the clock are used as the initial sequence number (also k bits). Each connection starts numbering its TPDUs with a different initial sequence number. The sequence space should be so large that by the time sequence numbers wrap around, old TPDUs with the same sequence number are long gone.
3. Once both transport entities have agreed on the initial sequence number, any sliding window protocol can be used for data flow control.

Elements of Transport Protocols: Connection Establishment

A problem occurs when a host crashes. When it comes up again, its transport entity does not know where it was in the sequence space.

- To require the transport entities to be idle for T seconds after a recovery to let all old TPDU's die off. (In a complex internetwork, T may be large, so this strategy is unattractive.)
- To avoid requiring T sec of dead time after a crash, it is necessary to introduce a new restriction on the use of sequence numbers.

Elements of Transport Protocols:

Connection Establishment: The forbidden region

Restriction on the sequence numbers ?

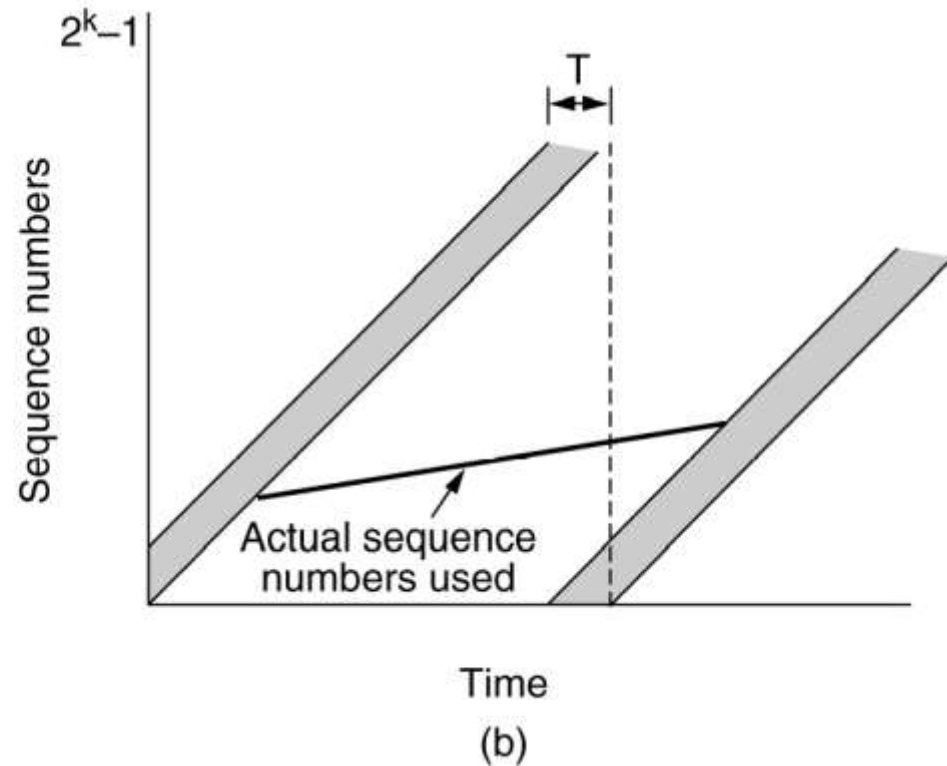
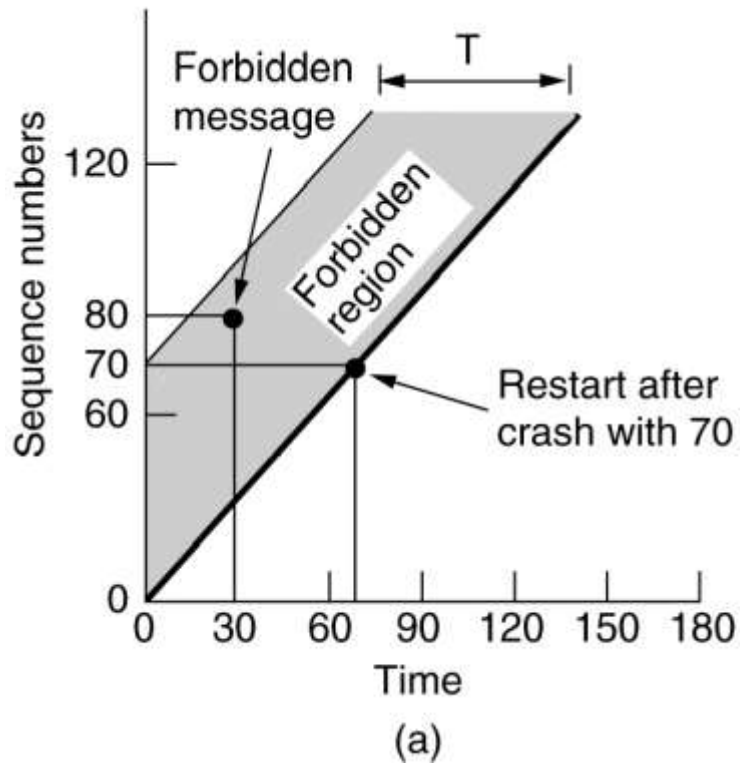
- Let T (the maximum packet lifetime) be 60 sec and let the clock tick once per second. The initial sequence number for a connection opened at time x will be x .
- At $t=30\text{sec}$, an ordinary data TPDU being sent on (a previously opened) connection 5 (called as TPDU X) is given sequence number 80.
- After sending TPDU X, the host crashes and then quickly restarts.
- At $t = 60\text{sec}$, it begins reopening connections 0 through 4.

Elements of Transport Protocols:

Connection Establishment: The forbidden region

- At $t = 70\text{sec}$, it reopens connection using initial sequence number 70 as required.
- During the next 15 sec it sends data TPDUs 70 through 80. Thus at $t=85\text{sec}$, a new TPDU with sequence number 80 and connection 5 has been injected into the subnet.
- ➔ TPDU X and TPDU 80.
- **To prevent sequence numbers from being used for a time T before their potential use as initial sequence numbers. The illegal combinations of time and sequence number are called as the forbidden region. Before sending any TPDU on any connection, the transport entity must read the clock and check to see that it is no in the forbidden region.**

Elements of Transport Protocols: Connection Establishment



Elements of Transport Protocols:

Connection Establishment: The forbidden region

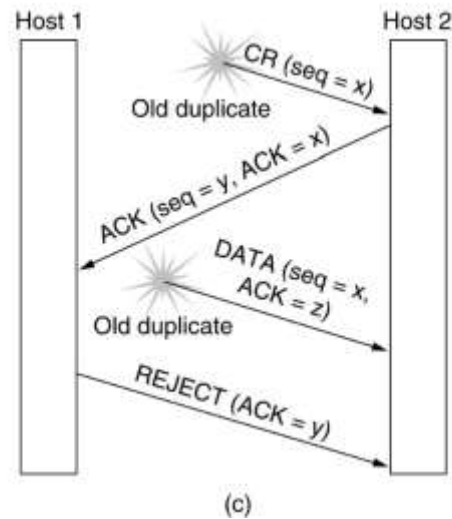
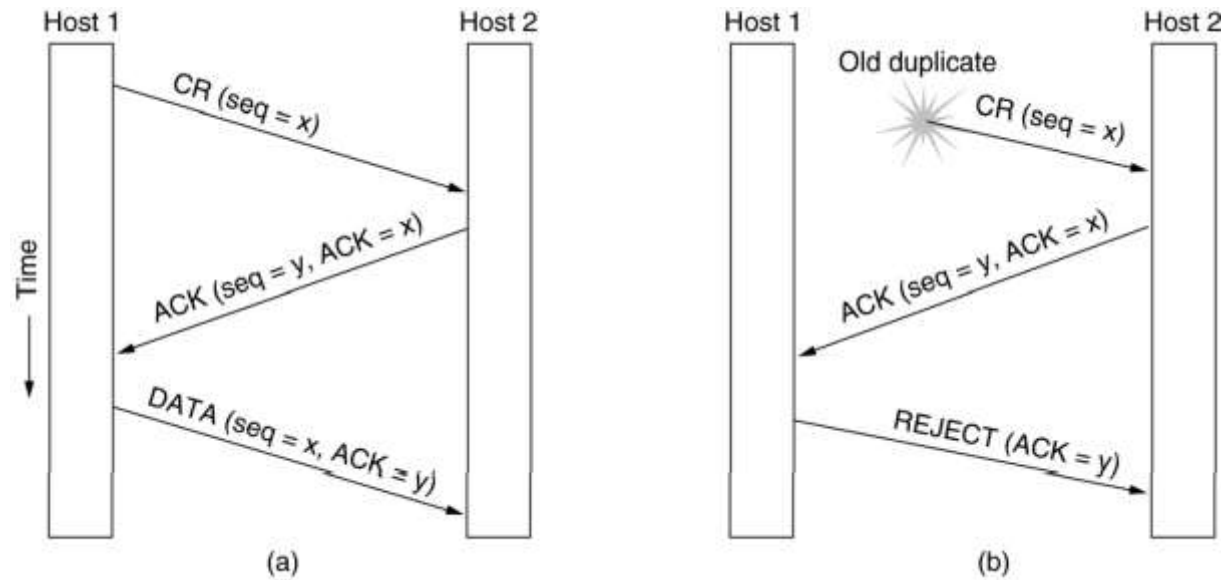
- Too fast
- Too slow
- **→ (Solution for the delayed TPDU)**
Before sending every TPDU,
the transport entity must check to see if it is about to
enter the forbidden region,
and if so, either delay the TPDU for T sec or
resynchronize the sequence numbers.

Elements of Transport Protocols:

Connection Establishment

- Three-way handshake for connection establishment
 - Normal operation.
 - Old duplicate CONNECTION REQUEST appearing out of nowhere.
 - Duplicate CONNECTION REQUEST and duplicate ACK.
- Conclusion: there is no combination of old CONNECTION REQUEST, CONNECTION ACCEPTED, or other TPDU's that can cause the protocol to fail and have a connection setup by accident when no one wants it.

Elements of Transport Protocols: Connection Establishment

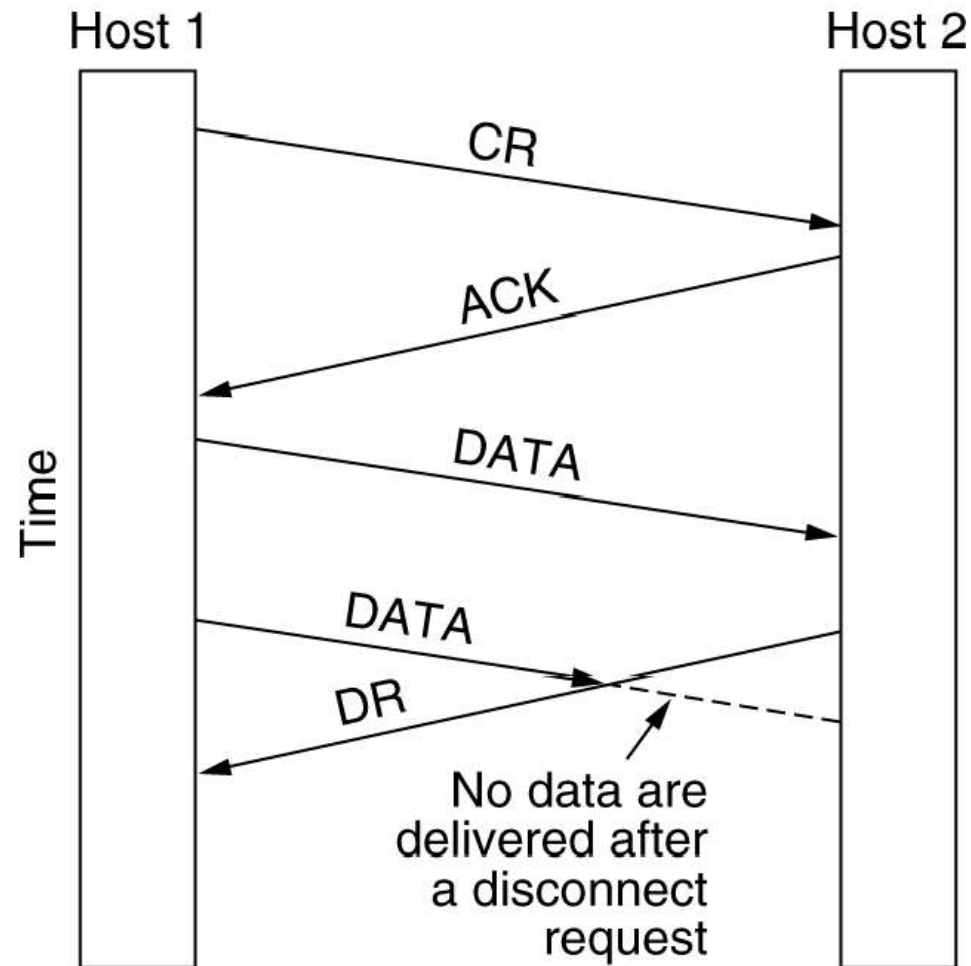


Elements of Transport Protocols: Connection Release

- Asymmetric release and symmetric release
 - Asymmetric release: When one part hangs up, the connection is broken.
 - Symmetric release: to treat the connection as two separate unidirectional connections and require each one to be released separately.
- Asymmetric release is abrupt and may result in data loss (See the next slide)
- One way to avoid data loss is to use symmetric release.

Elements of Transport Protocols: Connection Release

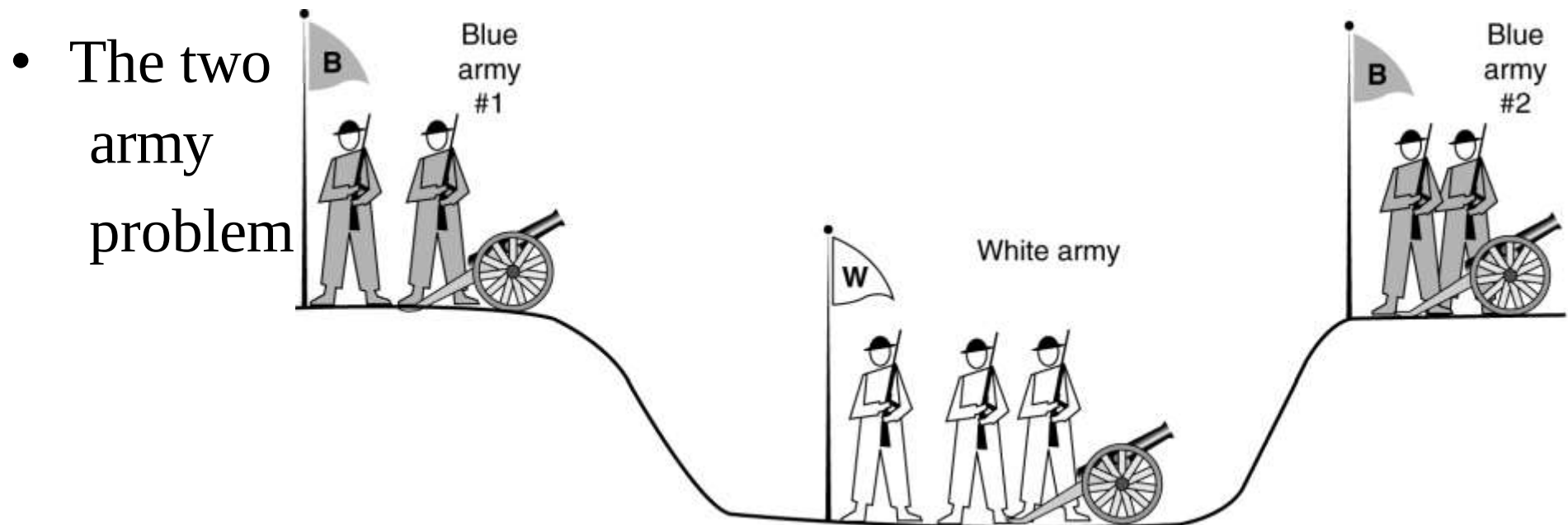
- Abrupt disconnection with loss of data



Elements of Transport Protocols:

Connection Release

- Symmetric release does the job when each process has a fixed amount of data to send and clearly knows when it has sent it.
- Symmetric release has its problems if determining that all the work has been done and the connection should be terminated is not so obvious.



Elements of Transport Protocols:

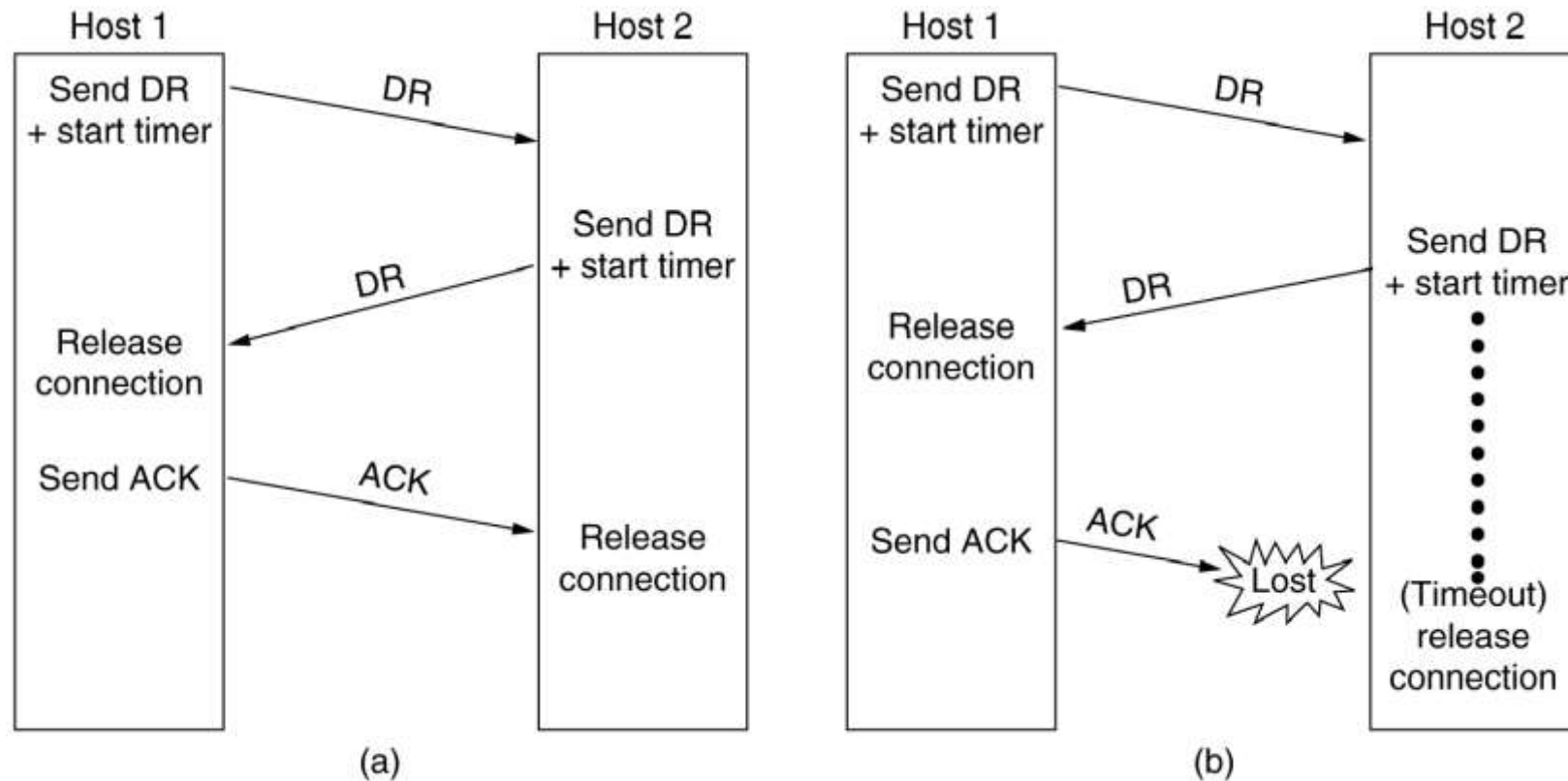
Connection Release

- Two army-problem: A white army is encamped in a valley. On both of the surrounding hillsides are blue armies.
one blue army < white army < two blue armies
- Does a protocol exist that allows the blue armies to win?
 - Two-way handshake
The commander of blue army #1: “I propose we attack at dawn on March 29. How about it?”
The commander of blue army #2: “OK.”
→ Will the attack happen?
 - Three-way handshake,
 - ...
 - N-way handshake
 - → No protocol exists that works.
- Substitute “disconnect” for “attack”. If neither side is prepared to disconnect until it is convinced that the other side is prepared to disconnect too, the disconnection will never happen.

Elements of Transport Protocols: Connection Release

(a) Normal case of three-way handshake

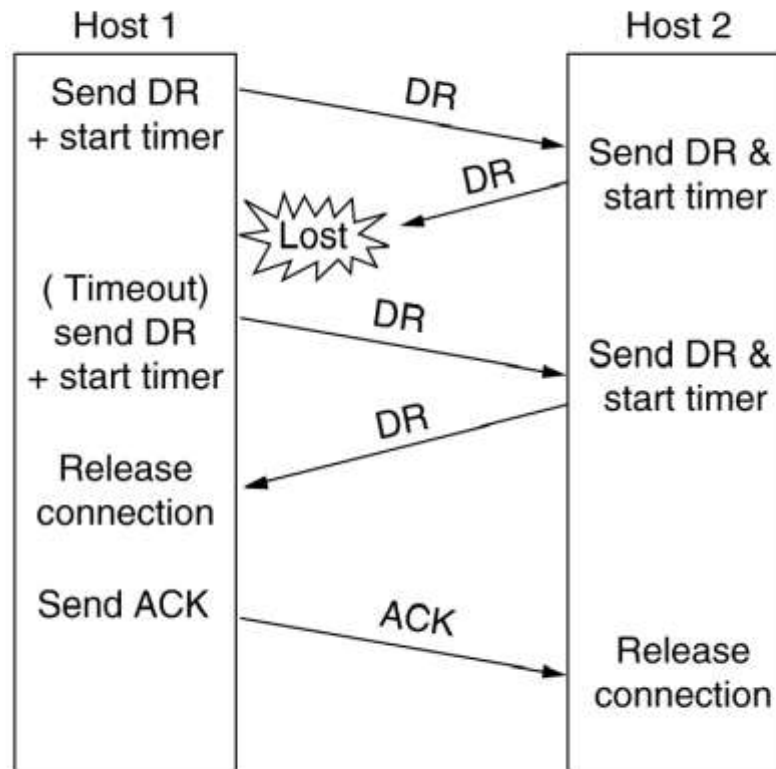
(b) Final ACK lost



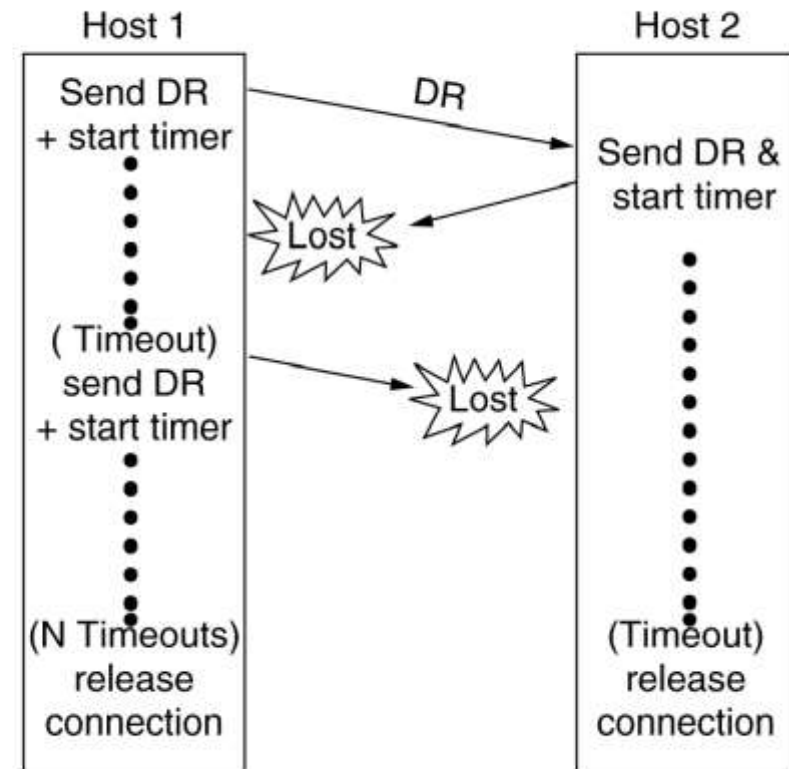
Elements of Transport Protocols: Connection Release

(c) Response lost

(d) Response lost and subsequent DRs Lost



(c)



(d)

Elements of Transport Protocols:

Connection Release

- Automatic disconnect rule
 - If no TPDUs have arrived for a certain number of seconds, the connection is then automatically disconnect.
 - Thus, if one side ever disconnects, the other side will detect the lack of activity and also disconnect.
- Conclusion: releasing a connection without data loss is not nearly as simple as it first appears.

Elements of Transport Protocols:

Error Control and Flow Control

Error control is ensuring that the data is delivered with the desired level of reliability, usually that all of the data is delivered without any errors.

- Similarity: In both layers, error control has to be performed.
- Difference: The link layer checksum protects a frame while it crosses a single link. The transport layer checksum protects a segment while it crosses an entire network path. It is an end-to-end check, which is not the same as having a check on every link.

Elements of Transport Protocols: Error Control and Flow Control

Flow control in data link layer and transport layer

- Similarity: In both layers a sliding window or other scheme is needed on each connection to keep a fast transmitter from overrunning a slow receiver.
- Difference: A router usually has relative few lines, whereas a host may have numerous connections.

Buffering

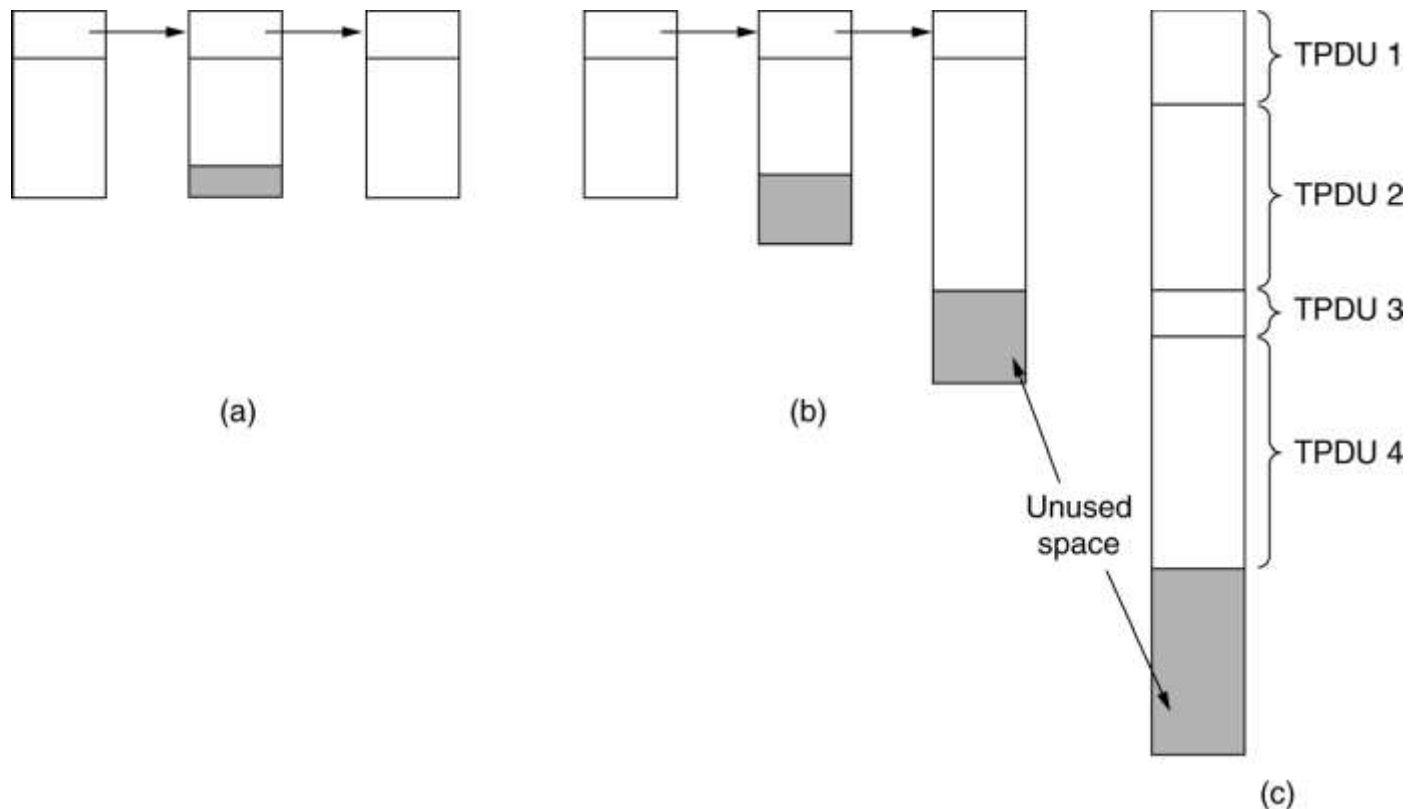
- The sender: The sender must buffer all TPDU's sent if the network service is unreliable. The sender must buffer all TPDU's sent if the receiver cannot guarantee that every incoming TPDU will be accepted.
- The receiver: If the receiver has agreed to do the buffering, there still remains the question of the buffer size.

Elements of Transport Protocols:

Flow control and buffering: (The receiver buffering)

Buffer sizes

- (a) Chained fixed-size buffers.
- (b) Chained variable-sized buffers.
- (c) One large circular buffer per connection.



Elements of Transport Protocols:

Flow control and buffering

- **The optimum trade-off between sender buffering and receiver buffering** depends on the type of traffic carried by the connection.
 - For low-bandwidth bursty traffic, it is better to buffer at the sender,
 - For high-bandwidth smooth traffic, it is better to buffer at the receiver.
 - As connections are opened and closed, and as the traffic pattern changes, the sender and receiver need to dynamically adjust their buffer allocation.
 - Dynamic buffer allocation.

Elements of Transport Protocols:

Flow control and buffering

Dynamic buffer allocation (receiver's buffering capacity).

	<u>A</u>	<u>Message</u>	<u>B</u>	<u>Comments</u>
1	→	< request 8 buffers>	→	A wants 8 buffers
2	←	<ack = 15, buf = 4>	←	B grants messages 0-3 only
3	→	<seq = 0, data = m0>	→	A has 3 buffers left now
4	→	<seq = 1, data = m1>	→	A has 2 buffers left now
5	→	<seq = 2, data = m2>	...	Message lost but A thinks it has 1 left
6	←	<ack = 1, buf = 3>	←	B acknowledges 0 and 1, permits 2-4
7	→	<seq = 3, data = m3>	→	A has 1 buffer left
8	→	<seq = 4, data = m4>	→	A has 0 buffers left, and must stop
9	→	<seq = 2, data = m2>	→	A times out and retransmits
10	←	<ack = 4, buf = 0>	←	Everything acknowledged, but A still blocked
11	←	<ack = 4, buf = 1>	←	A may now send 5
12	←	<ack = 4, buf = 2>	←	B found a new buffer somewhere
13	→	<seq = 5, data = m5>	→	A has 1 buffer left
14	→	<seq = 6, data = m6>	→	A is now blocked again
15	←	<ack = 6, buf = 0>	←	A is still blocked
16	...	<ack = 6, buf = 4>	←	Potential deadlock

Elements of Transport Protocols:

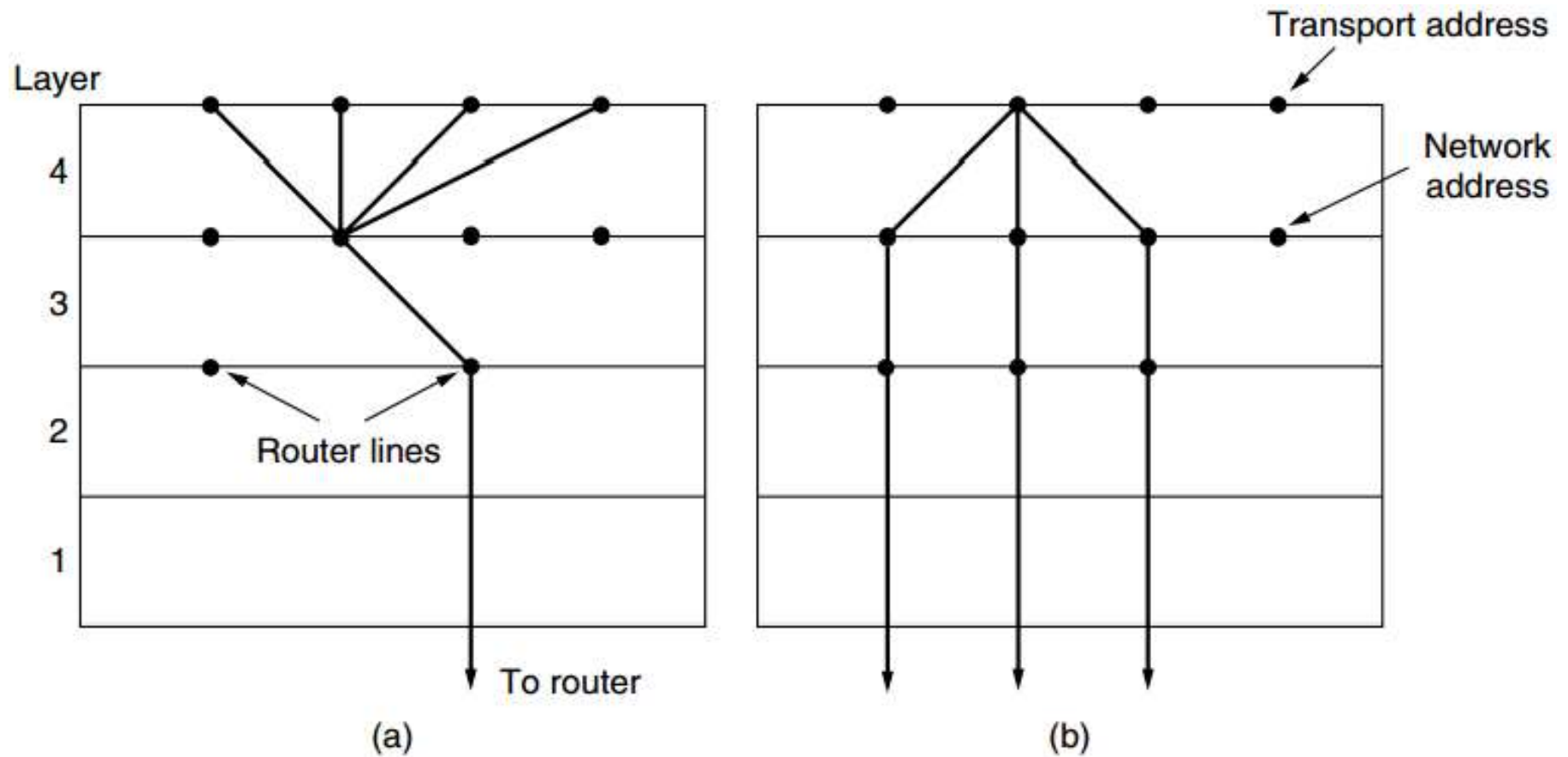
Flow control and buffering

- When buffer space no longer limits the maximum flow, another bottleneck will appear: the carrying capacity of the subnet.
 - The sender dynamically adjusts the window size to match the network's carrying capacity.
 - In order to adjust the window size periodically, the sender could monitor both parameters and then compute the desired window size.

Elements of Transport Protocols: Multiplexing

- Multiplexing and demultiplexing
 - Multiplexing: Application layer → Transport layer
→ Network layer → Data link layer → Physical layer
 - Demultiplexing: Physical layer → Data link layer
→ Network layer → Transport layer → Application layer
- Two multiplexing:
 - Upward multiplexing
 - Downward multiplexing (See the next slide)

Elements of Transport Protocols: Multiplexing



Elements of Transport Protocols: Crash Recovery

- Recovery from network and router crash is straightforward.
- Recovery from host crash is difficult.
- Assume that a client is sending a long file to a file server using a simple stop-and -wait protocol.
 - Part way through the transmission, the server crashes.
 - The server might send a broadcast TPDU to all other hosts, announcing that it had just crashed and requesting that its clients inform it of the status of all open connections.
 - Each client can be in one of two states: one TPDU outstanding, *S1*, or no TPDUs outstanding, *S0*.

Elements of Transport Protocols: Crash Recovery

- Some situations
 - The client should retransmit only if has an unacknowledged TPDU outstanding (S1) when it learns the crash.
 - If a crash occurs after the acknowledgement has been sent but before the write has been done, the client will receive the acknowledgement and thus be in state S0. → LOST. Problem!
 - If a crash occurs after the write has been done but before the acknowledgement has been sent, the client will not receive the acknowledgement and thus be in state S1. → Dup. Problem!
 - For more, see the next slide

Three events are possible at the server:

1. sending an acknowledgement (A),
2. writing to the output process (W),
3. and crashing (C).

Elements of Transport Protocols:

Crash Recovery

Strategy used by receiving host

First ACK, then write

First write, then ACK

Strategy used by
sending host

AC(W)

AWC

C(AW)

C(WA)

W AC

WC(A)

Strategy used by sending host	AC(W)	AWC	C(AW)	C(WA)	W AC	WC(A)
Always retransmit	OK	DUP	OK	OK	DUP	DUP
Never retransmit	LOST	OK	LOST	LOST	OK	OK
Retransmit in S0	OK	DUP	LOST	LOST	DUP	OK
Retransmit in S1	LOST	OK	OK	OK	OK	DUP

OK = Protocol functions correctly

DUP = Protocol generates a duplicate message

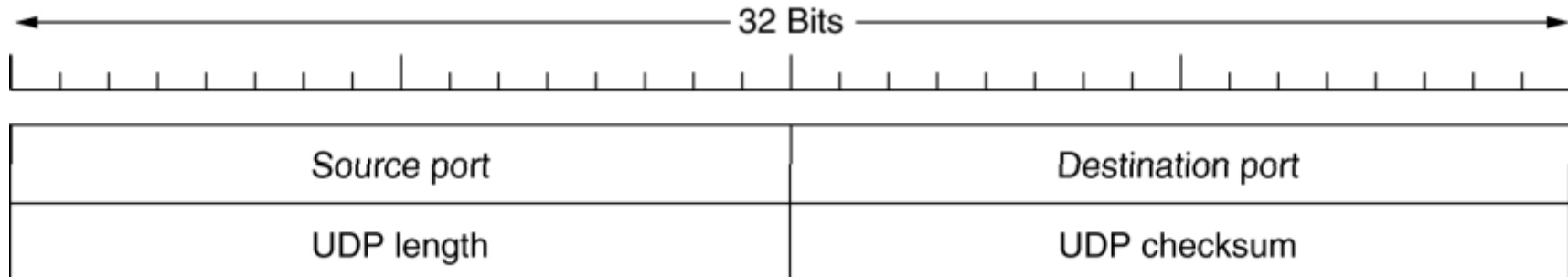
LOST = Protocol loses a message

Elements of Transport Protocols: Crash Recovery

- No matter how the sender and receiver are programmed, there are always situations where the protocol fails to recover properly.
- **In more general terms, recovery from a layer N crash can only be done by layer N+1 and only if the higher layer retains enough status information.**

The Internet Transport Protocols: UDP

- **UDP (RFC768)** provides a way for applications to
 - send encapsulated IP datagrams and
 - send them without having to establish a connection.
- UDP transmits segments consisting of an 8-byte header followed by the payload.



■ UDP

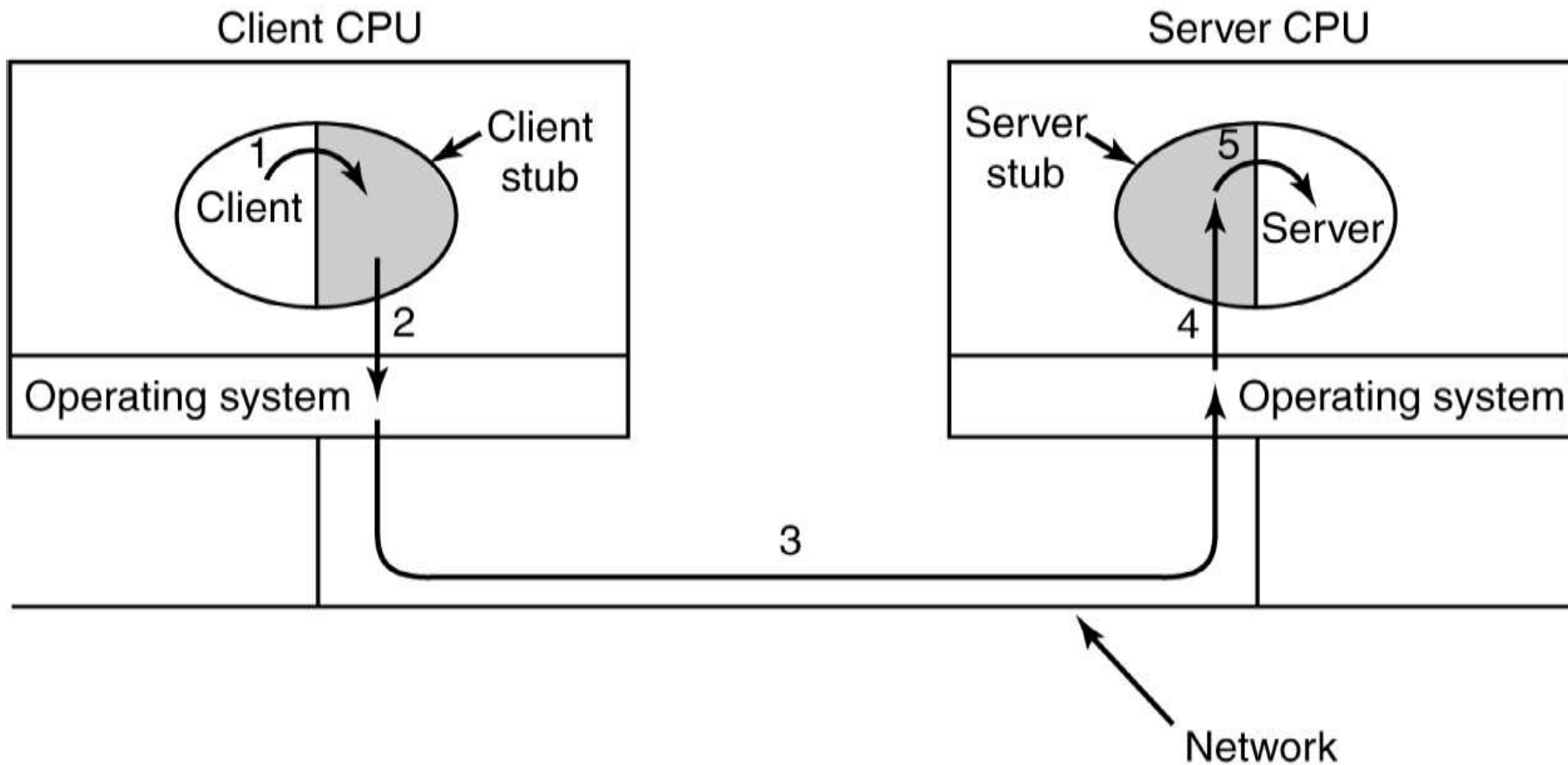
- ◆ Multiplexing and demultiplexing using ports.
- ◆ No flow control, error control or retransmission.
- ◆ Applications: RPC, RTP, DNS (Domain Name System)

The Internet Transport Protocols: RPC/UDP

- RPC (Remote Procedure Call) allows programs to call procedures located on remote hosts.
 - When a process on machine 1 calls a procedure on machine 2, the calling process on 1 is suspended and execution of the called procedure takes place on 2
 - Information can be transported from the caller to the callee in the parameters and can come back in the procedure result.
 - No message passing is visible to the programmer.
- **The idea behind RPC is to make a remote procedure call look as much as possible like a local one.**

C Proc $\longleftrightarrow$ C Stub $\longleftrightarrow$ S Stub $\longleftrightarrow$ S Proc

The Internet Transport Protocols: RPC/UDP



The Internet Transport Protocols: RPC/UDP

The steps in making an RPC

- Step 1 is the client calling the client stub.
- Step 2 is the client stub packing the parameters into a message (marshaling) and making a system call to send the message.
- Step 3 is the kernel sending the message from the client machine to the server machine.
- Step 4 is the kernel passing the incoming packet to the server stub and unpacking the packet to extract the parameters (unmarshaling).
- Step 5 is the server stub calling the server procedure with the unmarshaled parameters.
- The reply traces the same path in the other direction.

The Internet Transport Protocols: RPC/UDP

- A few snakes hiding under the grass (RPC)
 - The use of pointer parameters.
 - Some problems for weakly-typed languages (The length of an array).
 - It is not always possible to deduce the types of the parameters, not even from a formal specification or the code itself. (printf)
 - The use of global variables.
- → **Some restrictions are needed to make RPC work well in practice.**
- **RPC/TCP vs RPC/UDP**

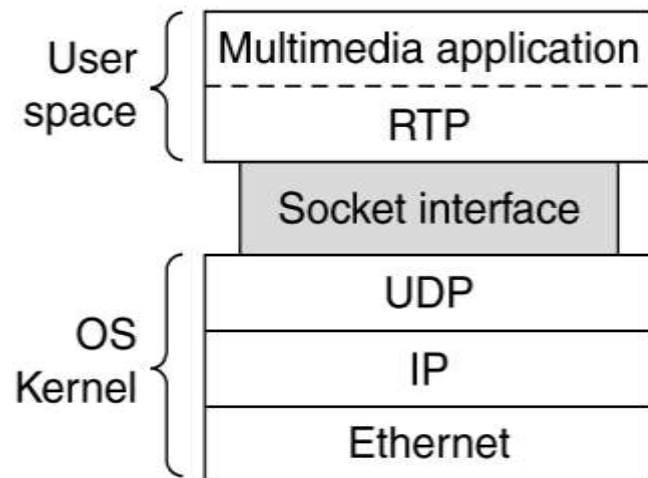
The Internet Transport Protocols: RTP/UDP

- Multimedia applications such as Internet radio, Internet telephony, music-on-demand, videoconferencing, video-on-demand, require real-time transport protocols. → RTP (Real-time Transport Protocol) (RFC1889)
- The RTP is in user space and runs over UDP. The RTP Ops:
 - The multimedia applications consists of multiple audio, video, text, and possibly other streams. These are fed into the RTP library.
 - This library then multiplexes the streams and encodes them in RTP packets, which it then puts into a socket.
 - UDP packets are generated and embedded in IP packets.
 - The IP packet are then put in frames for transmission.
 - ...

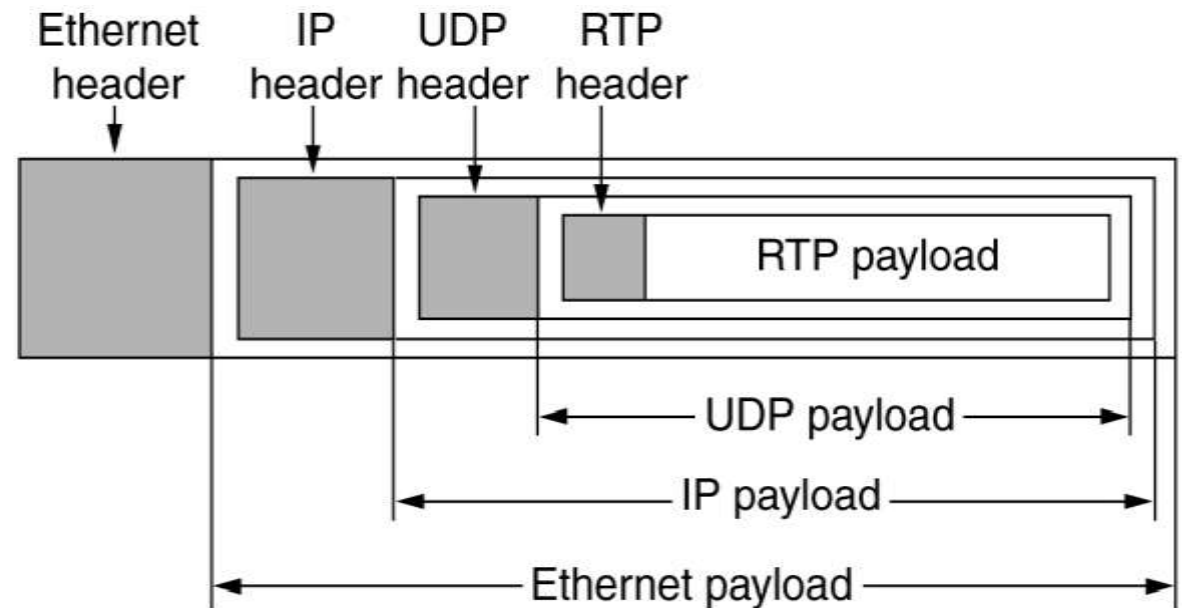
The Internet Transport Protocols: RTP/UDP

(a) The position of RTP in the protocol stack

(b) packet nesting



(a)

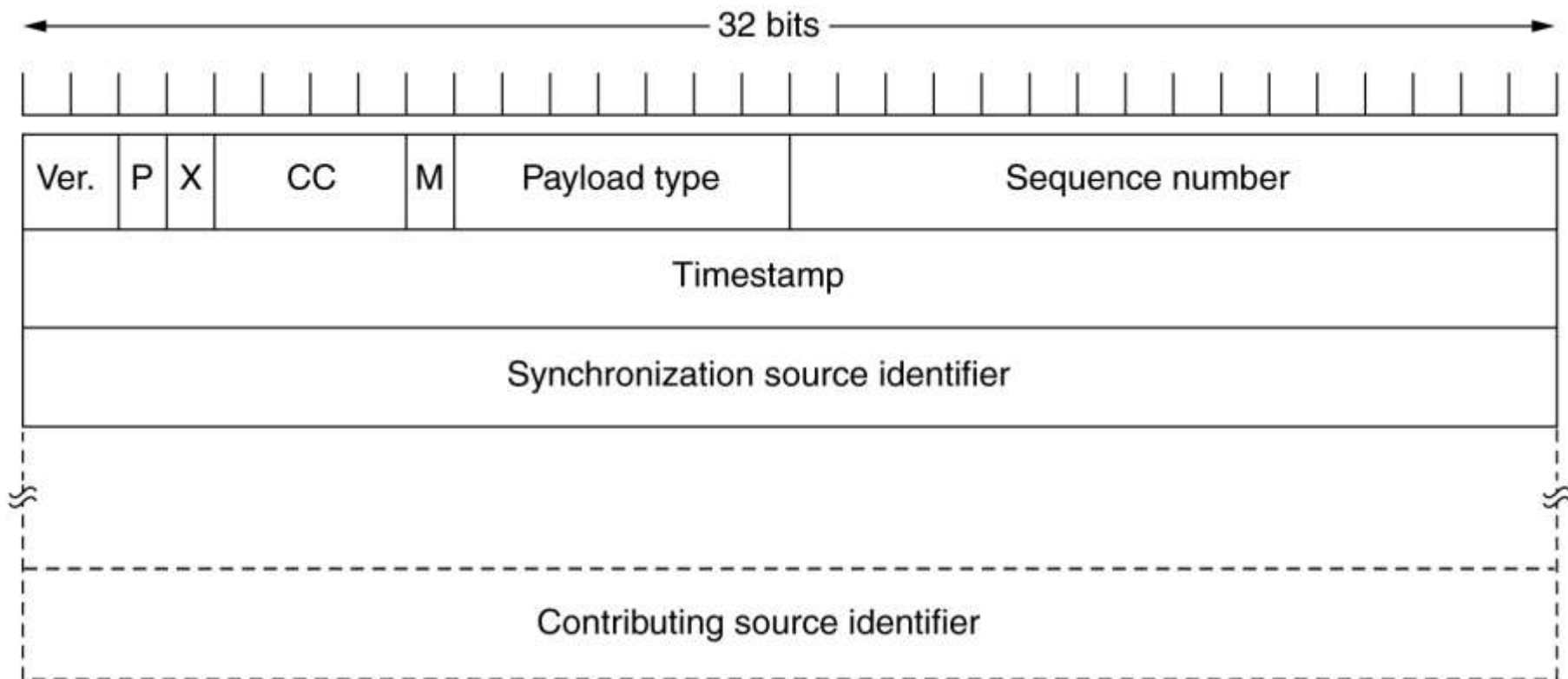


(b)

The Internet Transport Protocols: RTP/UDP

- RTP is a transport protocol realized in the application layer.
- RTP is to multiplex several real-time data streams onto a single stream of UDP packets and unicast or multicast the UDP packets.
- Each RTP packet is given a number one higher than its predecessor. RTP has no flow control, no error control, no acknowledgements, and no retransmission support.
- Each RTP payload may contain multiple samples and they can be coded any way that the application wants. For example a single audio stream may be encoded as 8-bit PCM samples at 8kHz, delta encoding, predictive encoding, GSM encoding, MP3, and so on.
- RTP allows timestamping.

The Internet Transport Protocols: RTP/UDP



The Internet Transport Protocols: RTP/UDP

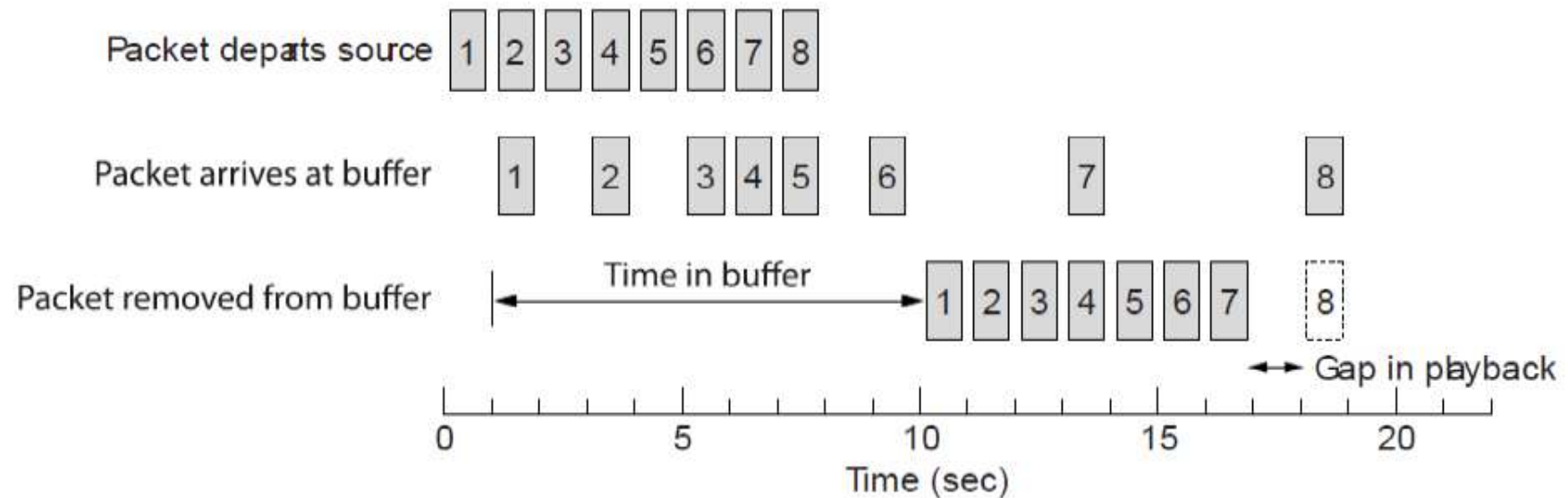
- **Version:** 2 bits, already at 2.
- **P bit:** padded to a multiple of 4 bytes or not.
- **X bit:** an extension header or not.
- **CC:** how many contributing sources are present (0-15).
- **M bit:** marker bit.
- **Payload type:** which encoding algorithm has been used.
- **Sequence number:** incremented on each RTP packet sent.
- **Timestamp:** reducing jitter.
- **Synchronization source identifier:** which stream the packet belongs to.
- **Contributing source identifiers.**

The Internet Transport Protocols: RTP/UDP

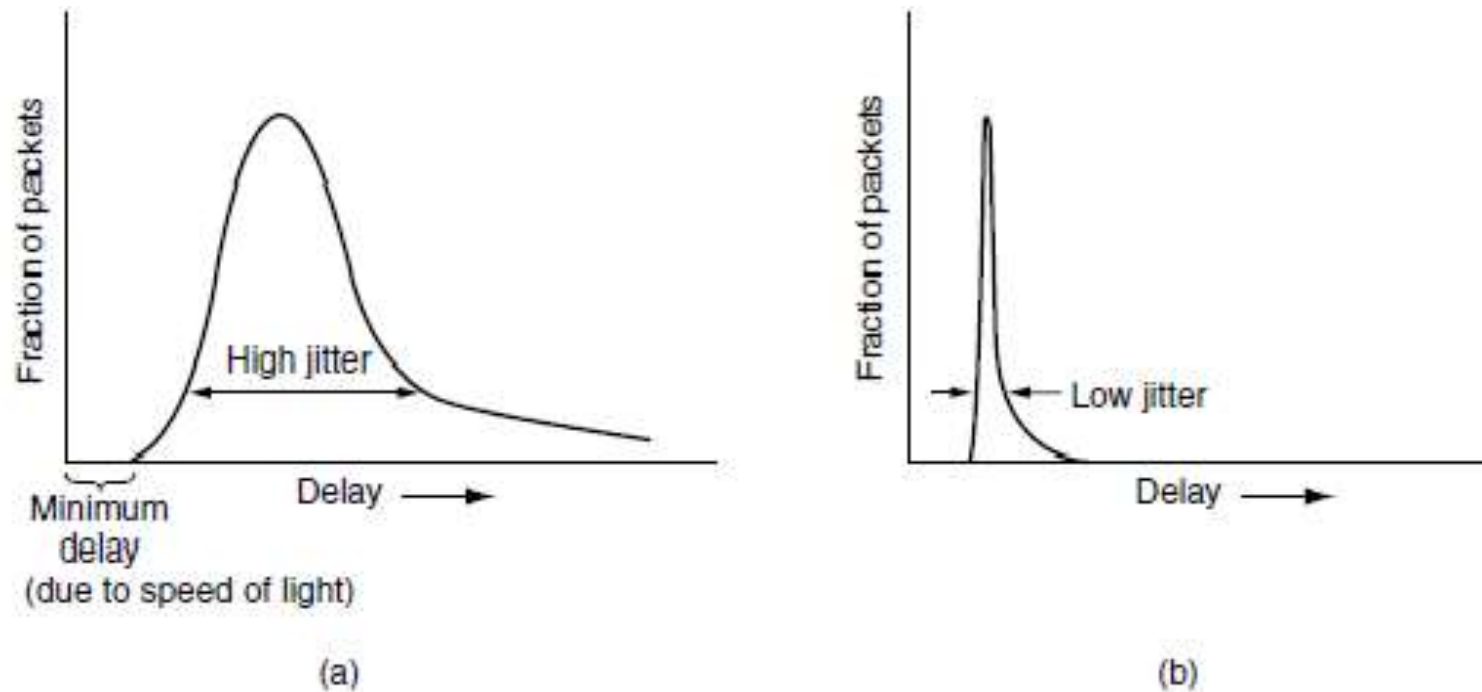
- RTCP (Real-time Transport Control Protocol) is a little sister protocol (little sibling protocol?) for RTP.
 - Does not transport any data
 - To handle feedback, synchronization, and the user interface
 - To handle interstream synchronization.
 - To name the various sources.
- For more information about RTP, *RTP: Audio and Video for the Internet* (Perkins, C.E. 2002, Addison-Wesley)

The Internet Transport Protocols: RTP/UDP

Smoothing the output stream by buffering packets



The Internet Transport Protocols: RTP/UDP



(a) High jitter. (b) Low jitter.